

Invited: Algorithm and Hardware Co-Design for Energy-Efficient Neural SLAM

Lingyi Huang*
Cheng Yang*
lingyi.huang@rutgers.edu
cheng.yang@rutgers.edu
Rutgers University
Piscataway, New Jersey, USA

Xiao Zang
Rutgers University
Piscataway, New Jersey, USA
xiao.zang@rutgers.edu

Yu Gong
Rutgers University
Piscataway, New Jersey, USA
yu.gong@rutgers.edu

Anthony Goeckner
Northwestern University
Evanston, Illinois, USA
anthony.goeckner@northwestern.edu

Bo Yuan
Rutgers University
Piscataway, New Jersey, USA
bo.yuan@rutgers.edu

Yang Sui
Rutgers University
Piscataway, New Jersey, USA
yang.sui@rutgers.edu

Qi Zhu
Northwestern University
Evanston, Illinois, USA
qzhu@northwestern.edu

ABSTRACT

In this paper, we introduce a novel approach to enhancing neural network-based Simultaneous Localization and Mapping (SLAM) through the integration of model compression techniques and customized hardware architecture that focuses on micro-architectural and dataflow optimizations to improve computational efficiency and performance. Experiments across different scenarios demonstrate that the proposed approach achieves significant improvement.

CCS CONCEPTS

• Computer systems organization → Embedded hardware.

KEYWORDS

Co-design, algorithm, hardware architecture, VLSI, robotics, SLAM

ACM Reference Format:

Lingyi Huang, Cheng Yang, Yu Gong, Yang Sui, Xiao Zang, Anthony Goeckner, Qi Zhu, and Bo Yuan. 2024. Invited: Algorithm and Hardware Co-Design for Energy-Efficient Neural SLAM. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3649329.3664829>

1 INTRODUCTION

Simultaneous Localization and Mapping (SLAM) plays a pivotal role in robotics, serving as the foundational technology that enables autonomous systems to navigate and comprehend their surroundings

with precision. The process involves the simultaneous construction of a map of the environment while also determining the location of the robot within that map. Recently, with the advent of deep learning, the use of neural networks to address SLAM challenges has gained popularity [8], offering promising avenues for enhancing SLAM performance and robustness.

However, the computational intensity and energy demands of neural network-based SLAM systems pose significant challenges, particularly for energy-constrained applications such as mobile robots and drones. In this paper, we introduce an algorithm and hardware co-design strategy that aims at enhancing the energy efficiency and computational performance of the neural SLAM module proposed in [1]. At the algorithm level, we propose a combined method of channel pruning and self-distillation built upon the existing model framework, which significantly reduces the parameters of the Neural SLAM feature extraction module while maintaining minimal degradation of accuracy. At the hardware level, a customized hardware architecture is proposed to support the Neural SLAM. This architecture focuses on three main optimizations: eliminating redundant computations by reusing data, facilitating frame-level parallel processing, and implementing an efficient trigonometric function look-up to reduce memory usage while ensuring fast and accurate calculations.

2 BACKGROUND

SLAM solves two intertwined problems: localization, which determines the robot's position within an environment, and mapping, which involves constructing a map of the environment itself. Visual SLAM, characterized by its reliance on camera data as the primary sensory input [3], has gained prominence due to the affordability and ease of integrating camera systems into various platforms [12].

Traditional Visual SLAM approaches [7][9] result in poor performance in difficult environments [8]. Deep learning emerges as a

*Both authors contributed equally to this research.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3664829>

powerful alternative [5, 6, 11], harnessing the capability to automatically discern and learn intricate features from visual inputs. Within this innovative domain, learning-based active SLAM [2][1] has attracted considerable attention. This method enables autonomous systems to actively and independently navigate and explore their surroundings, reducing the need for human guidance. The advantages of adopting learning-based active SLAM are manifold. It offers flexibility in using standard RGB images for spatial understanding, improvement of robustness to state estimation errors via adaptive learning, and the ability to leverage environmental patterns for better navigation in unfamiliar settings [1].

3 ALGORITHM

3.1 Problem Statement

Neural SLAM is a neural networks-based model specifically designed for SLAM tasks. The Neural SLAM f_θ processes the current RGB images, $s_t \in \mathbb{R}^{H \times W \times C}$, alongside the sensor readings of the agent's pose obtained from the current and last moment, $\hat{x}_{t-1:t}$, as well as the last estimates of the agent's pose and map, $\hat{x}_{t-1}$, m_{t-1} , to produce an updated map, m_t , and an estimate of the agent's current pose, $\hat{x}_t$. It can be expressed with a formula as follows:

$$m_t, \hat{x}_t = f_\theta(s_t, x_{0:t-1}, \hat{x}_{t-1}, m_{t-1}), \quad (1)$$

where θ are the trainable parameters of the Neural SLAM. However, the current Neural SLAM is characterized by its substantial model size and significant computational resource requirements. To address this issue, we aim to identify a more lightweight model based on the existing neural SLAM framework, which maintains performance with minimal loss, designated as model f_p . The parameters in model f_p should be notably less than the original model f_θ .

3.2 Methods

To obtain the lightweight model f_p from f_θ , model pruning is a common method. However, these approaches predominantly involve end-to-end fine-tuning to discover important weight parameters that are more relevant to the specific task [10]. For tasks based on Neural SLAM, the visual perception is mainly located in the front-end of SLAM, and employing global and local optimizations for end-to-end fine-tuning is exceedingly superfluous. In neural networks, the gradient of a parameter can be seen as a reflection of the sensitivity of the parameter to the task. A gradient-based pruning method with self-distillation [4] was proposed, selecting weights in the original model f_θ that are most critical for the SLAM task to avoid end-to-end fine-tuning during the pruning phase.

To identify the weight parameters sensitive to the current Neural SLAM task, we employ the original model to serve as the teacher model f_θ , and designate the original model with added Gaussian noise as the student model $f_{\theta+\epsilon}$. Thus, for the same input x , f_θ and $f_{\theta+\epsilon}$ produce different feature maps which construct a knowledge distillation loss function for self-distillation. By performing back-propagation on the loss, we can identify which parameters are most sensitive based on their gradients. We use $A \in \mathbb{R}^{C \times H \times W}$ to denote the feature map produced by the teacher or student model, where C, H, W denotes its channel number, height, and width, respectively, which can be formulated as:

$$A^T = f_\theta(x), \quad (2)$$

$$A^S = f_{\theta+\epsilon}(x), \quad (3)$$

$$L_{KD} = \sum_{k=1}^C \sum_{i=1}^H \sum_{j=1}^W \|A_{k,i,j}^T - A_{k,i,j}^S\|_2, \quad (4)$$

where ϵ represents Gaussian noise of the same dimension as the weight parameters. Consider L_{KD} to be the loss function, and let $\mathbb{W} \in \mathbb{R}^{C_o \times C_i \times H \times W}$ represent a convolutional kernel with C_o output channels, C_i input channels, height H , and width W . Then we calculate the sum of the absolute gradients for each convolutional kernel in every layer. The formula can be simplified to the following:

$$G_{c_o}^i = \sum_{c_i=1}^{C_i} \sum_{h=1}^H \sum_{w=1}^W \left| \frac{\partial L_{KD}}{\partial W_{c_o, c_i, h, w}} \right|, \quad (5)$$

where $G_{c_o}^i$ represents the sum of absolute gradients for the c_o^{th} convolutional kernel in the i^{th} layer.

3.3 Algorithm Workflow

Initially, we obtain the teacher model f_θ and the student model $f_{\theta+\epsilon}$, which has added Gaussian noise ϵ to f_θ . We sample 10000 examples and use the loss from knowledge distillation L_{KD} to perform back-propagation, tallying the cumulative absolute gradient information of each convolutional layer's kernels. In the student model $f_{\theta+\epsilon}$, throughout the pruning phase, we prioritize the retention of the upper 50% of kernels in the feature extraction module. This strategic selection culminates in the derivation of the pruned student model f_p . Subsequently, f_θ serves as the ground truth for the training of the student model f_p . Upon completion of this training phase, the student model f_p is integrated into the entire SLAM end-to-end pipeline, followed by a fine-tuning process spanning 100 epochs to optimize the pipeline's performance. The algorithm workflow is summarized in Algorithm 1.

Algorithm 1 Model Pruning Based on Gradient Information - Utilizing Self-Distillation.

Input: original SLAM model f_θ , gaussian noise ϵ , data sample D from Gibson dataset(T)

- 1: obtain gradients from L_{KD}
- 2: **for** $d = 1$ to D **do**
- 3: calculate L_{KD} from f_θ and $f_{\theta+\epsilon}$
- 4: obtain gradients (G) based on L_{KD} backpropagation
- 5: **end for**
- 6: obtain the pruned model(f_p) based on gradients(G)
- 7: learn parameters θ in f_p with L_{KD}
- 8: **for** $d = 1$ to T **do**
- 9: calculate L_{KD} from f_θ and f_p
- 10: L_{KD} backward
- 11: optimizer parameters θ
- 12: **end for**
- 13: finetune the pruned model(f_p) in original Neural SLAM pipeline

Output: final pruned model f_p

4 HARDWARE DESIGN

Building upon the algorithm in Section 3, we propose a specialized hardware architecture for accelerating Neural SLAM through data reuse, parallel processing, and efficient trigonometric look-ups.

4.1 Elimination of Redundant Computations.

Pose estimation in Neural SLAM heavily relies on the continuous transformation of projections to refresh the egocentric frame of reference. In the original Neural SLAM module, this method requires repeated computation of the egocentric map prediction from the most recent RGB observation at each iteration. This repetitive recalibration of the egocentric map demands a significant portion of the system’s computational resources, exacerbating the energy consumption and processing latency for real-time SLAM.

To address this inefficiency, we propose an approach that diverges from the traditional computation-heavy paradigm. As shown in Fig. 1 (Middle), by storing the egocentric map generated at a given time step within a dedicated memory space, sized at 16 KB to accommodate the spatial map size up to 240×240 , we enable the system to retrieve this data for subsequent pose estimations. This method leverages the continuity between successive observations to minimize redundant computations.

4.2 Enable Frame-level Parallel Processing

Following our strategy to eliminate redundant computations through data reuse, we identify that the sequential processing of neural network forwarding and spatial transformations remains an impediment to achieving real-time performance. We propose a strategy for frame-level parallel processing, as depicted in Fig. 1 (Bottom), to enable the concurrent execution of neural network operations and spatial transformations. Capitalizing on the independence of these tasks, we distribute them across distinct hardware units (see Section 4.4 for details) for parallel processing. This strategic realignment not only maximizes hardware utilization but also reduces processing delays.

4.3 Efficient Trigonometric Function Look-up

To further enhance our Neural SLAM hardware’s efficiency, we implement the Efficient Trigonometric Function Look-up mechanism, which is crucial for spatial transformations. This method significantly reduces memory usage by storing trigonometric values only for angles within $[0, \pi/4]$, leveraging trigonometric symmetry and periodicity to cover all possible angles with one-eighth of the typical storage.

As shown in Fig. 2, upon receiving an angle, a pre-processing circuit maps it into our stored range. The look-up module then retrieves the sine and cosine values for this mapped angle. A post-processing circuit adjusts these values to produce the original angle’s sine and cosine, utilizing simple logic to reflect the trigonometric functions’ properties across the entire $[-\pi, \pi]$ range. This method significantly reduces memory requirements while ensuring accurate and rapid trigonometric computations.

4.4 Overall Architecture

The overall hardware architecture is shown in Fig. 3. The accelerator takes the RGB observation and the agent’s poses from the

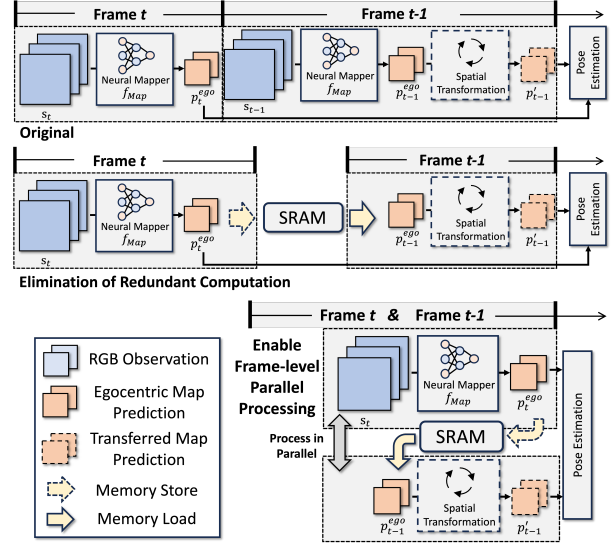


Figure 1: (Top) Original Computation Flow. (Middle) Elimination of Redundant Computations. (Bottom) Enable Frame-level Parallelism.

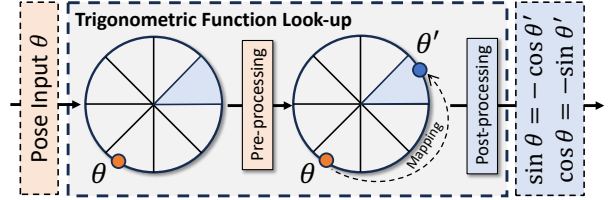


Figure 2: Trigonometric Function Look-up.

sensor, incorporating both current and previous time steps, as inputs. Firstly, the observation is processed by the MAC array, which performs the neural network forwarding operations. The result, the map prediction for the current time step p_t^{ego} , is stored at the Pred. SRAM for further reuse. Concurrently, the sensor reading of the pose, along with the last frame’s egocentric map prediction p_{t-1}^{ego} , are input into the ST (Spatial Transformation) Engine. Within this Engine, trigonometric functions related to the pose are retrieved from the Look-up Module and used by an address generator to facilitate the reconstruction from p_{t-1}^{ego} to p_{t-1}' through the Reconstruction Unit. Afterwards, both the current (p_t^{ego}) and the newly transformed (p_{t-1}') predictions are sent to the MAC array for neural pose estimation computation. Subsequently, the ST engine transforms the p_t^{ego} to m_t^{ego} based on the predicted pose $\hat{x}$. The final stage is to aggregate the m_t^{ego} with the previous map m_{t-1} from the Map SRAM using a channel-wise pooling process executed by a comparator array.

5 EVALUATION

5.1 Algorithm Evaluation

To evaluate the performance of all models regarding scene size, we divided the Gibson validation set into two subsets: a smaller

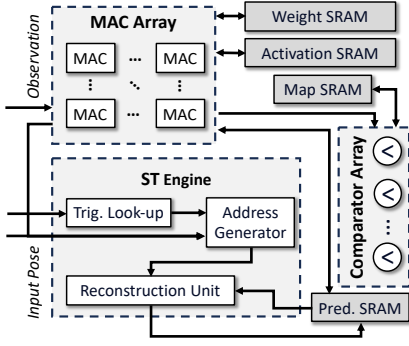


Figure 3: Overall Architecture.

collection comprising 10 scenes with explorable areas ranging from 16m^2 to 36m^2 , and a larger collection consisting of 4 scenes, where the explorable areas vary from 55m^2 to 100m^2 . It is important to note that the overall size of the maps significantly exceeds the traversable areas, with the largest map extending approximately 23 meters in length and 11 meters in width.

A point is considered traversable and known if it falls within the agent’s field of view and is no more than 3.2 meters distant. We adopted two evaluation metrics: the absolute coverage area in square meters (Cov) and the percentage of the scene explored (% Cov), which is the ratio of the covered area to the maximum possible coverage within a given scene. Throughout the training phase, each episode is conducted over a fixed duration of 1000 steps. We show the experiment results of the baseline Neural SLAM and our models in Table 1.

Table 1: Experiment Results of Neural SLAM and Our Models

Method	Parameters(M)	%Cov.	Cov.(m^2)
Baseline	19.29	94.7	31.75
Pruning	13.22(↓ 31.4%)	50.7	15.35
Pruning + KD	13.22(↓ 31.4%)	82.7	25.87
Pruning + KD + Finetune(f_p)	13.22(↓ 31.4%)	88.1	28.18
Pruning + KD + Finetune(all)	13.22(↓ 31.4%)	92.56	30.16

Finetune(f_p) denotes the process of training exclusively applied to the f_p model. Finetune(all) signifies the training applied across the entire pipeline of SLAM.

5.2 Hardware Evaluation

Accelerator Configuration. We showcase the effectiveness of our proposed hardware design through a case study on a neural SLAM hardware accelerator, designed to manage spatial maps up to 240×240 . The architecture incorporates 256 16-bit multipliers and utilizes 968.7 KB of on-chip memory.

Experimental Methodology. A high-level functional simulator was developed to model the hardware accelerator behavior. The architecture was developed using Verilog HDL and the RTL model was synthesized through Synopsys Design Compiler, employing CMOS 28nm technology. The design’s logical components’ area and power consumption were deduced from the synthesis report, alongside on-chip SRAM performance metrics provided by CACTI.

Hardware Performance. At an operational frequency of 1000 MHz, our design example occupies 2.36mm^2 and consumes 296.8 mW. Fig. 4 compares the performance of our hardware accelerator with the GPU-based software implementations, assessed on an NVIDIA RTX A6000 GPU with 1410 MHz clock frequency. Compared with GPU implementation, our accelerator achieves $67.6\times$ improvement on area efficiency and $13.8\times$ improvement on energy efficiency.

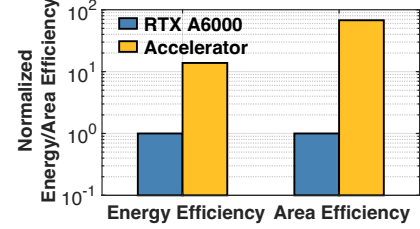


Figure 4: Hardware performance compared with GPU.

6 CONCLUSION

This paper develops an algorithm and hardware co-design strategy for enhancing the energy efficiency and computational performance of neural SLAM systems. The approach includes optimizing the algorithm to minimize parameter count and designing hardware architecture to reduce redundant computations, enable parallel processing, and streamline trigonometric calculations.

7 ACKNOWLEDGEMENT

This work is supported in part by National Science Foundation (NSF) Awards 2239945 and 2324936.

REFERENCES

- [1] Devendra Singh Chaplot, Dhiraj Gandhi, Saurabh Gupta, Abhinav Gupta, and Ruslan Salakhutdinov. 2020. Learning to explore using active neural slam. *arXiv preprint arXiv:2004.05155* (2020).
- [2] Tao Chen, Saurabh Gupta, and Abhinav Gupta. 2019. Learning exploration policies for navigation. *arXiv preprint arXiv:1903.01959* (2019).
- [3] Margarita N Favorskaya. 2023. Deep Learning for Visual SLAM: The State-of-the-Art and Future Trends. *Electronics* 12, 9 (2023), 2006.
- [4] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).
- [5] Peter Karkus, Shaojun Cai, and David Hsu. 2021. Differentiable slam-net: Learning particle slam for visual navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2815–2825.
- [6] Ruihao Li, Sen Wang, and Dongbing Gu. 2020. DeepSLAM: A robust monocular SLAM system with unsupervised deep learning. *IEEE Transactions on Industrial Electronics* 68, 4 (2020), 3577–3587.
- [7] David G Lowe. 2004. Distinctive image features from scale-invariant keypoints. *International journal of computer vision* 60 (2004), 91–110.
- [8] Saad Mokssit, Daniel Bonilla Licea, Bassma Guermah, and Mounir Ghogho. 2023. Deep Learning Techniques for Visual SLAM: A Survey. *IEEE Access* 11 (2023), 20026–20050.
- [9] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. 2011. ORB: An efficient alternative to SIFT or SURF. In *2011 International conference on computer vision*. Ieee, 2564–2571.
- [10] Yang Sui, Miao Yin, Yi Xie, Huy Phan, Saman Zonouz, and Bo Yuan. 2021. CHIP: CHannel Independence-based Pruning for Compact Neural Networks. In *Advances in Neural Information Processing Systems*.
- [11] Nan Yang, Lukas von Stumberg, Rui Wang, and Daniel Cremers. 2020. D3vo: Deep depth, deep pose and deep uncertainty for monocular visual odometry. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 1281–1292.
- [12] Jong-Hyeok Yoon and Arijit Raychowdhury. 2020. NeuroSLAM: A 65-nm 7.25-to-8.79-TOPS/W mixed-signal oscillator-based SLAM accelerator for edge robotics. *IEEE Journal of Solid-State Circuits* 56, 1 (2020), 66–78.

Invited: Using Causal Information to Aid Robots in Generalizing to New Tasks

Semanti Basu
Brown University
USA

Semir Tatlidil
Brown University
USA

Moon Hwan Kim
Brown University
USA

Steven Sloman
Brown University
USA

R. Iris Bahar
Colorado School of Mines
Brown University
USA

Abstract

Causal reasoning is a key factor that allows human beings to effortlessly draw parallels between two seemingly different tasks. Given access to a limited number of human-generated causal models, if robots could be taught to perform a similar “transfer”, then they could potentially generalize to different domains using limited human intervention. The goal is to effectively generate human-like causal models for an unknown task by learning from a small dataset of human-generated causal models on other related tasks. In this paper we propose two different methods to leverage causal models obtained from humans to generalize to a new task. We ran a user study to obtain causal models on how different light-producing objects function. We then explore one-to-one transfer, where we directly use a causal model generated by a participant to infer the causal model of another object. We also explore many-to-one transfer, where we leverage causal models from different objects and users to infer the causal model of an unknown object. Automated transfer of causal models from humans to unrelated domains has the potential to replicate human-like reasoning in an unknown scenario in the absence of an expert. That chain of reasoning represented through causal models can then be used by autonomous agents to guide several downstream tasks such as object assembly, trouble shooting, and repair.

ACM Reference Format:

Semanti Basu, Semir Tatlidil, Moon Hwan Kim, Steven Sloman, and R. Iris Bahar. 2024. Invited: Using Causal Information to Aid Robots in Generalizing to New Tasks. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3649329.3663514>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3663514>

1 Introduction and Related Work

Generalization is a core problem in computer science and has been addressed in different ways, such as domain adaptation, domain generalization, transfer learning, and meta learning [11]. Our approach to exploring the transferability of human-generated causal models can be considered a domain generalization problem. It is essentially the ability of models trained on source data to generalize to out-of-distribution data or unseen domains.

This concept of generalization is prevalent in electronic design automation (EDA) as well. In order to automate the modern chip design process, it is important to develop models that can handle distribution shift to generalize to previously unobserved circuit designs. In Chipformer [5], a decision-transformer based approach is introduced to learn transferable placement policies by utilizing offline reinforcement learning. In [9], chip floorplanning is framed as a Markov Decision process. A transferable representation of chips is learned through the use of graphical neural networks to generalize to unseen chips. PruneX [12] addresses generalization in the logic synthesis stage of chip design. Because of their graphical representation, causal models can prove to be an intuitive way of addressing generalization in placement or logic synthesis, which also follows a graphical layout. Moreover, capturing the mental models of hardware engineers through causal models can provide valuable insights into the reasoning behind the design process — something that is not necessarily captured by collecting placement behaviors for offline training, as done in [5]. Causal models can inform effective transfer to unseen domains since reasoning should be invariant across domains.

While causal models have not been used for generalization in chip design as of yet, causal representation learning through structural causal models has been explored as a basis for domain generalization in vision based tasks such as image classification [7, 8]. In robotics, the use of causality to learn generalizable manipulation policies is explored in CREST [6]. CausalWorld [1] introduces a benchmark to explore causal structure learning and generalization in robot manipulation scenarios.

Prior approaches to generalization, both causal and non-causal, attempt to learn a generalizable policy to perform the same task in a different setting [5, 6] or to learn an invariant representation that is expected to remain the same across a particular category/class [7–9]. In contrast, we attempt to decouple reasoning about an object from the task that is to be performed with the object. We collect human-generated causal models to explain how an object works. We then attempt to use the reasoning captured through causal models from humans to infer a causal model to explain how a new object works. These models can then be used to facilitate any task using these objects such as assembly, repair or troubleshooting. Our approach is general, and we hypothesize it may be useful in the context of EDA design and verification as well as robot reasoning and task planning.

In this paper, we propose two different approaches to achieve task-agnostic transfer of causal models obtained from humans to explain the working of an unknown object assuming that its part descriptions are given:

- *One-to-one approach*: Given a causal model relating object parts for the source object, we infer the causal model for a destination object by encoding the causal links in the source object and searching for the closest match in the destination object. This approach is similar to the work on binary causal question-answering, [3], which uses a k -nearest neighbor search together with cosine similarity to answer causal questions. However, that approach depends on the use of causal cues in sentences such as “cause” to generate data and does not deal with a full causal model (only cause-effect pairs in a sentence).
- *Many to one approach*: We aggregate the data obtained from our survey of human participants to elicit causal models and re-frame transfer as a probabilistic text classification problem. We learn from multiple different objects and user responses to infer the causal model of an unknown object. This is similar to binary causal question answering using BERT [4]; however, we do not include causal cues, which were central to the data extraction in [4]. Moreover, we use soft labels rather than hard labels.

2 Method

We ran a user study with 100 participants where we showed each of them 4 different objects — flashlight, desk lamp, kerosene lamp, and wall lamp. The objects parts and corresponding function descriptions are provided. For each pair of parts, the participants were asked questions of the form: “Remove [Part X]. Would the [Part Y] still perform [function of Y]?”. The causal model of the user for the object was inferred from all their answers.

2.1 On-to-one transfer

In this approach we characterize each node (denoting an object part) in the source causal graph using a vector embedding of the textual function description of the part. The vector embedding from text can be obtained from any Large

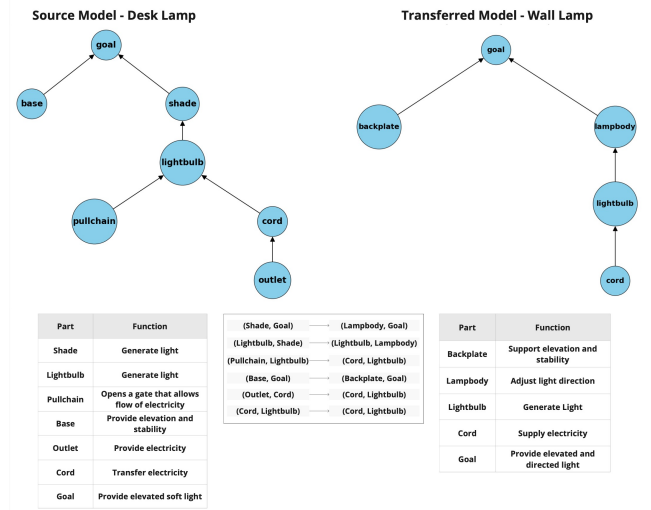


Figure 1. One-to-one transfer

Language Model (LLM) based encoder. We used the popular Sentence-Transformer library to generate the embeddings (in our case a numerical vector of length 768). To represent a causal relationship between two object parts, we concatenate the embeddings from the “cause” and “effect” nodes, with the “cause” vector preceding the “effect” vector to embed the direction.

For the new object to which we want to transfer the source causal model, we assume that the parts and their descriptions are known. Each object part is a node characterized by the vector embedding of its function description. We want to infer the causal relationships between the parts by using the source causal model as a prior. To that end, we generate all pairwise permutations of parts for the new object and their corresponding edge embeddings by concatenating node embeddings. For each causal link in the source model, we find the edge embedding in the destination object that to which it is most similar. That edge embedding is inferred to be causal in the destination object. We use cosine similarity as a distance metric to measure closeness of embeddings. An example transfer is shown in Fig. 1.

2.2 Many-to-one transfer

In this approach, we reframe causal model transfer as a text classification problem. In our causal model extraction approach from humans, a series of pairwise interventional questions were asked and the corresponding binary Yes/No answers were recorded. We aggregate this information from the survey to create our dataset (Fig. 2). Each question together with the context forms a single entry in our dataset with two soft labels Yes and No. The values of the labels are the percentages of people surveyed who answered Yes and No to that question respectively. We then finetuned a pre-trained BERT model [2] on our data for probabilistic text classification and added a dropout layer and a linear layer on top of the pre-trained BERT model (bert-base-uncased) to

Query	Yes	No
A Kerosene lamp provides smokeless light. It consists of the following object parts: chimney, wick, burner, kerosene tank, and lighter. The Chimney prevents smoke. The Wick carries fuel to the flame. The Burner controls the size of the flame. The Kerosene tank provides fuel. The Lighter starts a fire. Imagine that the chimney is removed from the kerosene lamp. Remove the chimney. Would the wick still carry fuel to the flame?	1	0
A Kerosene lamp provides smokeless light. It consists of the following object parts: chimney, wick, burner, kerosene tank, and lighter. The Chimney prevents smoke. The Wick carries fuel to the flame. The Burner controls the size of the flame. The Kerosene tank provides fuel. The Lighter starts a fire. Imagine that the chimney is removed from the kerosene lamp. Remove the chimney. Would the burner still control the size of the flame?	0.94949495	0.05050505
A Kerosene lamp provides smokeless light. It consists of the following object parts: chimney, wick, burner, kerosene tank, and lighter. The Chimney prevents smoke. The Wick carries fuel to the flame. The Burner controls the size of the flame. The Kerosene tank provides fuel. The Lighter starts a fire. Imagine that the chimney is removed from the kerosene lamp. Remove the chimney. Would the kerosene tank still provide fuel?	1	0
A Kerosene lamp provides smokeless light. It consists of the following object parts: chimney, wick, burner, kerosene tank, and lighter. The Chimney prevents smoke. The Wick carries fuel to the flame. The Burner controls the size of the flame. The Kerosene tank provides fuel. The Lighter starts a fire. Imagine that the chimney is removed from the kerosene lamp. Remove the chimney. Would the lighter still start a fire?	0.98989899	0.01010101

Figure 2. Dataset used

create our custom baseline model. We also created another model where we add three convolutional layers and a ReLU layer on top of a pre-trained BERT model before adding a linear layer to map it to outputs. Finally, we created yet another model similar to our baseline model but we replace the standard pre-trained BERT model with a BERT model fine-tuned on natural language inference data [13] to test the effects of transfer learning on our dataset and task.

Cross-entropy loss was used for training. Since there were four objects, we tested transfer by training our custom model on the data related to three objects and tested it on the fourth. We used cross-validation to evaluate transfer across different kinds of objects. We set aside a small fraction of the training dataset to be used for validation and used early stopping to prevent-overfitting while training. We further used ensemble modeling by training BERT 11 times for each split of the data and then taking a majority vote at inference time to mitigate biases from random initial seeds. We further followed guidelines from [10] while setting other parameters. At inference, all interventional questions for the unseen object were asked of the trained model. The model then output probabilities for Yes and No. If the probability for No exceeded 0.5, a cause-effect relationship is considered to exist, otherwise no.

We chose to use BERT instead of an LLM in order to learn solely from the mental models of the users surveyed instead of drawing from an external repository of knowledge. However, we benchmarked the performance of ChatGPT and LLaMA on our dataset in order to evaluate how they perform on our dataset. In order to evaluate these LLMs, we directly asked them all the same interventional questions we asked of the users in the survey and recorded their answers (Yes/No). We then inferred the causal model from their answers. We also finetuned GPT-3.5 and LLaMA using our user data and recorded the answers from the finetuned models as well. We finetuned GPT-3.5-Turbo by integrating our dataset directly into the OpenAI platform. It is crucial to highlight that altering the model’s internal weights directly is not allowed and is an automatic process. Instead, we honed its output for binary (Yes or No) responses by finetuning the inference temperature to 0.3, which yielded the most effective results. For LLaMA, the finetuning process involved the use of the “meta-llama/llama-2-13b-chat-hf” model, employing techniques such as Low-Rank Adaptation (LoRA) to modify only a subset of the model’s parameters, thus ensuring

Source Object	Transferred Object	Avg. Transferred model F1
Desk	Flash	0.558322455
	Wall	0.619971751
	Kerosene	0
Flash	Desk	0.500078172
	Wall	0.469
	Kerosene	0
Kerosene	Flash	0.240722833
	Desk	0.31191564
	Wall	0.319753358
Wall	Desk	0.378759035
	Flash	0.435776279
	Kerosene	0
Average:		0.31952496

Table 1. One to one Transfer

efficiency and retaining the pre-trained model’s extensive knowledge base. One of the primary challenges encountered was ensuring that the model’s output remained within the confines of Yes/No answers, requiring careful adjustment of the training dataset and finetuning parameters. Consequently, we had to create a specific formatting function to ensure the prompts and completions are structured in a way that directly gives us Yes/No answers as a safety measure.

3 Experimental Results

3.1 Evaluating one-to-one transfer

In order to evaluate the quality of causal models obtained from participants as well as through transfer, we developed our own ground truth causal models for each object (*i.e.*, causal models generated ourselves). We compare causal models to these ground truth models to evaluate them. We use a metric called *F1 score* to assess how close a model is to the ground truth. If a causal link is present in a causal model (user-generated or obtained through transfer) as well as in the ground truth causal model for that object, it is considered a true positive. If a causal link is present in a causal model but not in the ground truth model, it is considered a false positive. If a causal link is not present in the causal model but present in the ground truth model, it is counted as a false negative. For the one-to-one transfer procedure, we performed transfer for all user models across each pair of objects. The average F1 score across all users for each type of transfer is reported in Table 1.

3.2 Evaluating many-to-one transfer

In Table 2 we present results from our many-to-one experiments using different types of BERT architectures. It can be

seen that the Baseline BERT model outperforms the model that had CNN layers on top of pre-trained BERT. Pre-training on the MNLI dataset led to degradation in performance.

The Baseline BERT model achieves similar performance (average F1 score = 0.61) to average user models across all objects (average F1 score = 0.65). The aggregated subject user model outperforms it by a significant margin (F1 score = 0.76). Note that the aggregated subject model is formed by combining answers from all users surveyed to infer the model, so its superior performance is expected because it aggregates the knowledge and reasoning of 100 different people.

Model	Desk	Flash	Kero	Wall	Avg.
Baseline BERT	0.67	0.73	0.36	0.67	0.61
BERT+CNN	0.50	0.50	0.36	0.67	0.51
BERT MNLI pretrained	0.13	0.55	0.18	0.55	0.35
Average Subject	0.64	0.82	0.47	0.69	0.65
Aggregated Subject	0.74	1.00	0.53	0.75	0.76
ChatGPT - GPT4	0.83	0.75	0.17	0.50	0.56
ChatGPT Untrained 3.50	0.67	0.57	0.41	0.40	0.51
ChatGPT Finetuned 3.5	0.67	0.20	0.35	0.00	0.30
LLaMA Pre-Finetuning	0.22	0.44	0.38	0.46	0.38
LLaMA Finetuned	0.36	0.25	0.48	0.25	0.33

Table 2. F1 score for many-to-one transfer

It is interesting to note that ChatGPT 4 performed quite well for common objects such as desk lamp and flashlight but very poorly for the lesser known kerosene lamp. This is suggestive of the fact that GPT-4 likely relied on general knowledge rather than reasoning to achieve its performance. On average, our baseline BERT model outperformed ChatGPT-4 and LLaMA. It can also be seen in Table 2 that finetuning worsened performance for both ChatGPT3.5 and LLaMA.

4 Discussion and Conclusion

We explored the transfer of causal models of different objects obtained from humans to explain how a new object works. We showed that fine-tuning BERT on a task of probabilistic text classification using our crowdsourced data provides robust transfer results comparable to actual models obtained from humans. Transfer learning from natural language inference datasets and using large language models degraded the performance contrary to expectations. In the future, we hope to explore strategies for better transfer through the use of contrastive learning for more effective transfer from related tasks, thereby allowing robots and other autonomous systems to complete a greater range of tasks.

5 Acknowledgement

This work was sponsored by the Office of Naval Research (ONR), under grant #N000142212494. The views and conclusions contained herein are those of the authors only and should not be interpreted as representing those of ONR, the U.S. Navy or the U.S. Government.

References

- [1] O. Ahmed, F. Träuble, A. Goyal, A. Neitz, M. Wüthrich, Y. Bengio, B. Schölkopf, and S. Bauer. Causalworld: A robotic manipulation benchmark for causal structure and transfer learning. In *International Conference on Learning Representations*, 2021.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, pages 4171–4186, June 2019.
- [3] O. Hassanzadeh, D. Bhattacharjya, M. Febowitz, K. Srinivas, M. Perrone, S. Sohrabi, and M. Katz. Answering binary causal questions through large-scale text mining: An evaluation using cause-effect pairs from human experts. In *International Joint Conference on Artificial Intelligence, IJCAI-19*, pages 5003–5009. International Joint Conferences on Artificial Intelligence Organization, 7 2019.
- [4] H. Kayesh, Md. Saiful I., J. Wang, S. Anirban, A.S.M. Kayes, and P. Waters. Answering binary causal questions: A transfer learning based approach. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–9, 2020.
- [5] Y. Lai, J. Liu, Z. Tang, B. Wang, J. Hao, and P. Luo. ChiPFormer: Transferable chip placement via offline decision transformer. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18346–18364. PMLR, 23–29 Jul 2023.
- [6] T. E. Lee, J. A. Zhao, A. S. Sawhney, S. Girdhar, and O. Kroemer. Causal reasoning in simulation for structure and transfer learning of robot manipulation policies. In *IEEE International Conference on Robotics and Automation*, pages 4776–4782, 2021.
- [7] F. Lv, J. Liang, S. Li, B. Zang, C. H. Liu, Z. Wang, and D. Liu. Causality inspired representation learning for domain generalization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8046–8056, 2022.
- [8] D. Mahajan, S. Tople, and A. Sharma. Domain generalization using causal matching. In *International Conference on Machine Learning*, pages 7313–7324. PMLR, 2021.
- [9] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. M. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi, J. Pak, A. Tong, K. Srinivasa, W. Hang, E. Tuncer, Q. V. Le, J. Laudon, R. Ho, R. Carpenter, and J. Dean. A graph placement methodology for fast chip design. *Nature*, 594:207 – 212, 2021.
- [10] M. Mosbach, M. Andriushchenko, and D. Klakow. On the stability of fine-tuning bert: Misconceptions, explanations, and strong baselines. *arXiv preprint arXiv:2006.04884*, 2020.
- [11] J. Wang, C. Lan, C. Liu, Y. Ouyang, T. Qin, W. Lu, Y. Chen, W. Zeng, and P. Yu. Generalizing to unseen domains: A survey on domain generalization. *IEEE Transactions on Knowledge and Data Engineering*, 2022.
- [12] Z. Wang, L. Chen, J. Wang, X. Li, Y. Bai, X. Li, M. Yuan, J. Hao, Y. Zhang, and F. Wu. A circuit domain generalization framework for efficient logic synthesis in chip design. *arXiv preprint arXiv:2309.03208*, 2023.
- [13] A. Williams, N. Nangia, and S. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In M. Walker, H. Ji, and A. Stent, editors, *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, pages 1112–1122, June 2018.